Day 130 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 5 reference

Watch: Variational Autoencoders simplified (Prof. Ryan Ahmed) | Variational Autoencoder - VISUALLY EXPLAINED! (Kapil Sachdeva)

`day130_vae_latent_space.py`, segment by segment, in the order `main()` uses things. Encoding, decoding, sampling, interpolation and the two judging tools are new; training is Day 129's train_vae and the plain autoencoder is Day 126's train_ae, both imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: enc, dec = a trained VAE's two halves · mu, lv = code means and log-variances · Z = codes · ref = real images used as a yardstick · beta = the KL weight

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE); sys.path.insert(0, os.path.join(HERE, "..",
    "week-18-unsupervised-learning-i"))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day129_exercise as d129
    import day119_exercise as d119
ENC = ["relu", "linear"]; DEC = ["relu", "linear"]
AE = ["relu", "linear", "relu", "linear"]
```

- `import day129_exercise as d129` — Day 129's VAE parts: vae_loss and train_vae.
- `ENC = [...]; DEC = [...]` — two names on one line: the VAE's halves each have one hidden ReLU layer.

Segment 1 — Using a VAE

encode, decode, sample_prior, interpolate

MATH

$$z(t) = (1 - t)\, z_0 + t\, z_1, \quad t = 0, 1/8, \ldots, 1$$

CODE

```
def encode(enc, X):
    H = d126.forward(enc, X, ENC)[-1]; k = H.shape[1] // 2
    return H[:, :k], H[:, k:]
def decode(dec, Z):
    return d126.sigmoid(d126.forward(dec, Z.astype(np.float32), DEC)[-1])
def sample_prior(dec, n, k, rng):
    return decode(dec, rng.standard_normal((n, k)))
def interpolate(z0, z1, steps):
    t = np.linspace(0, 1, steps)[:, None]
    return (1 - t) * z0 + t * z1
```

- `H[:, :k], H[:, k:]` — the encoder's output split into means and log-variances.

- `Z.astype(np.float32)` — codes made in float64 (by NumPy's random generator) are cast to the network's float32.
- `np.linspace(0, 1, steps)[:, None]` — a column of fractions: broadcasting gives one code per row.
- `(1 - t) * z0 + t * z1` — the straight line from z0 (t = 0) to z1 (t = 1).

nearest_real and active_units

MATH

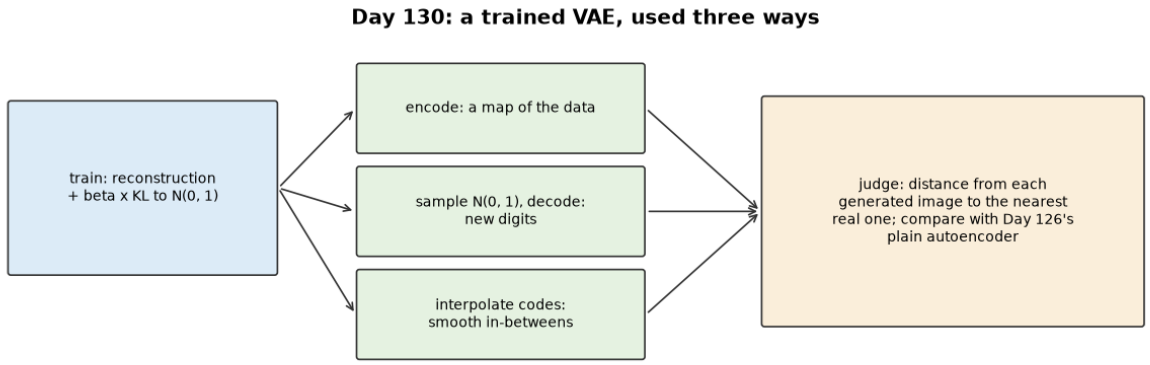
$$d(x) = \sqrt{(\min_r |x - r|^2 / 784)}$$

CODE

```
def nearest_real(R, ref, chunk=500):
    return np.concatenate([np.sqrt(np.maximum(d119.sqdist(R[s:s + chunk].astype(np.float64), ref),
        0).min(1) / R.shape[1])
                           for s in range(0, len(R), chunk)])
def active_units(enc, X, thresh=0.1):
    mu, lv = encode(enc, X); kl = (0.5 * (np.exp(lv) + mu ** 2 - 1 - lv)).mean(0)
    return int((kl > thresh).sum()), kl
```

- `np.concatenate([... for s in range(0, len(R), chunk)])` — in chunks of 500, so the distance table never exceeds 500 x 10,000.
- `.min(1) / R.shape[1]` — the nearest real image's squared distance, per pixel; the square root makes it a root-mean-square difference.
- `(kl > thresh).sum()` — how many dimensions carry more than 0.1 nats on average.

SYSTEM DIAGRAM



Encode, sample, interpolate, judge.

Segment 2 — The experiments

demo_generate: the two models' random codes

CODE

```
C = d126.forward(ae, X[:20000], AE)[2]; m, s = C.mean(0), C.std(0)
fake_ae = d126.sigmoid(d126.forward(ae[2:], (m + s * rng.standard_normal((1000,
    16))).astype(np.float32), AE[2:])[-1])
fake_vae = sample_prior(dec16, 1000, 16, rng)
```

- `d126.forward(ae[2:], ..., AE[2:])` — the plain autoencoder's decoder alone: its last two layers.

- `m + s * rng.standard_normal((1000, 16))` — random codes matched to the real codes' mean and spread, as on Day 126.

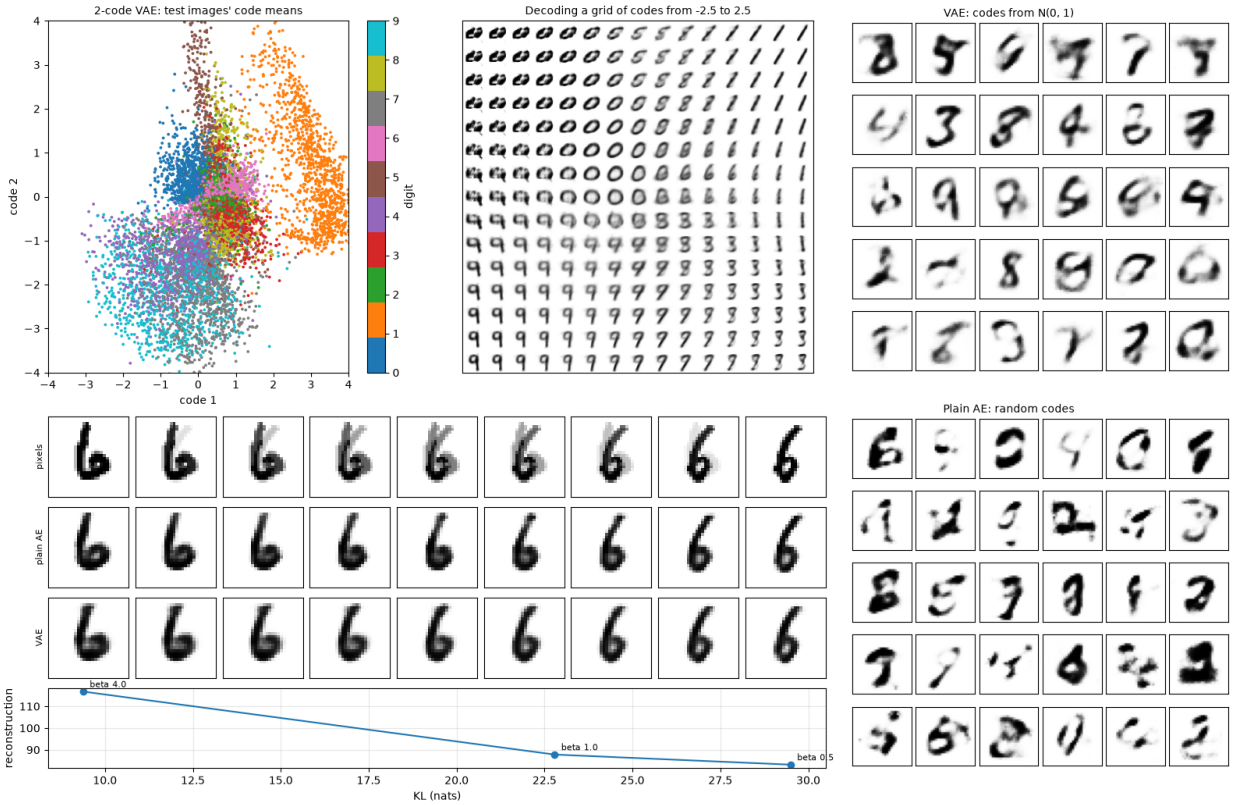
plot_results: decoding a grid of codes

CODE

```
a = fig.add_subplot(G[0, 1]); g = np.linspace(-2.5, 2.5, 15); Z = np.array([(u, v) for v in
    g[::-1] for u in g])
I = decode(dec2, Z).reshape(15, 15, 28, 28).transpose(0, 2, 1, 3).reshape(15 * 28, 15 * 28)
```

- `[(u, v) for v in g[::-1] for u in g]` — a nested comprehension: every grid point, top row first.
- `.reshape(15, 15, 28, 28).transpose(0, 2, 1, 3)` — 225 images rearranged so pixel rows of neighbouring images sit side by side.
- `.reshape(15 * 28, 15 * 28)` — one large image: the whole grid of decoded digits.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.linspace(0, 1, n)[:, None]` — a column of evenly spaced fractions.
- `[f(u, v) for v in a for u in b]` — a nested list comprehension.
- `a.transpose(0, 2, 1, 3)` — reorder axes.
- `net[2:]` — the last layers of a network on their own.
- `np.concatenate(list)` — join chunks back together.
- `x.astype(np.float32)` — change precision.

End of Day 130 syntax reference. For the experiments and what they showed, see the Day 130 complete guide.